

PHP Basics

Interaktive Medien 3

nick.schneeberger@fhgr.ch

Inhalt

- Was ist PHP?
- Variablen & Datentypen
- Funktionen
- Bedingungen
- Arrays
- Schleifen

Nach jedem Kapitel folgt ein Code-Along.

Zum Nachschlagen gibt es zu jedem Thema ein Cheatsheet im Ordner `cheatsheets/`.

Was ist PHP?

Die Sprache, die auf dem Server läuft

Was ist PHP?

- Eine **serverseitige Skriptsprache** für das Web
- Der Code läuft auf dem **Server**, nicht im Browser
- Dateien enden auf `.php`
- Der Browser bekommt nur das **Ergebnis** – nie den Code
- Typisch: Formulare verarbeiten, Datenbanken abfragen, Daten ausliefern

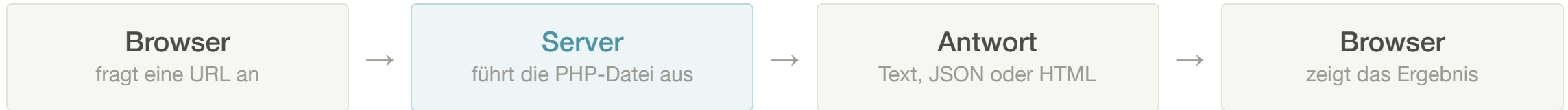
INDEX.PHP

```
<?php  
$message = "Hallo Welt!";  
echo $message;
```

IM BROWSER SICHTBAR

Hallo Welt!

Browser und Server



Deshalb musst du deine Datei über `http://...` aufrufen – also über den Kursserver oder `php -S localhost:8000`.

Ein Doppelklick öffnet `file://...`, dort läuft kein PHP und du siehst nur den Code als Text.

PHP und JavaScript

	JavaScript (IM2)	PHP (IM3)
Läuft	im Browser	auf dem Server
Sichtbar	Code ist einsehbar	nur das Ergebnis
Datenbank	nein	ja
Variablen	<code>let temp = 19.4</code>	<code>\$temp = 19.4;</code>
Datei	<code>index.html</code> + <code>script.js</code>	<code>index.php</code>

Vieles kennst du schon: Variablen, Funktionen, Bedingungen, Arrays und Schleifen gibt es in beiden Sprachen.

Vor allem die Schreibweise ist neu.

Die erste PHP-Datei

```
<?php
// Das ist ein Kommentar
$message = "Hallo PHP";
echo $message;
```

- `<?php` öffnet den PHP-Bereich
- Jede Anweisung endet mit **Semikolon** `;`
- `//` macht einen Kommentar
- `echo` schreibt einen Wert in die Antwort
- Das schliessende `?>` lassen wir weg, wenn die Datei nur PHP enthält

PHP und HTML lassen sich mischen.

In diesem Kurs machen wir das bewusst **nicht** – siehe nächste Folie.

PHP als Datenlieferant

In diesem Kurs bauen wir keine Webseite mit PHP.

PHP liefert nur **Daten** – wir kontrollieren sie mit `echo` und `var_dump`.

```
<?php
header('Content-Type: text/plain; charset=utf-8');    // reine Textausgabe

echo "Aare in Bern: 19.4 °C\n";
```

Variablen & Datentypen

Werte speichern und wiederverwenden

Variablen: die Regeln

- Beginnen immer mit `$`
- Erlaubt: Buchstaben, Zahlen, `_`
- Aber nie mit einer Zahl beginnen
- Gross-/Kleinschreibung zählt: `$name` $\neq$ `$Name`
- Zuweisung mit `=`
- **Dynamisch typisiert:** der Typ ergibt sich aus dem Wert

```
$location      = 'Bern';  
$temperatureC  = 19.4;  
$measuredAt    = '10:00';  
$isOfficial    = true;
```

Sprechende Namen in **camelCase**: `$temperatureC` statt `$t`.

Das zählt sich im Projekt aus.

Datentypen

```
$integer = 42;           // int    – Ganzzahl  
$float   = 19.4;        // float  – Kommazahl  
$string  = "Aare in Bern"; // string – Text  
$boolean = true;        // bool   – true oder false  
$nothing = null;        // null   – kein Wert  
$array   = [18.9, 19.4, 20.1]; // array – mehrere Werte
```

Ausgeben und prüfen

```
echo $temperatureC;  
  
var_dump($temperatureC);  
var_dump($isOfficial);  
  
print_r($temperatures);
```

AUSGABE

```
19.4  
float(19.4)  
bool(true)  
Array  
(  
    [0] => 18.9  
    [1] => 19.4  
)
```

- `echo` schreibt den **Wert** – für die Ausgabe
- `var_dump` zeigt zusätzlich **Typ und Struktur** – zum Debuggen
- `print_r` zeigt Arrays lesbar, aber ohne Typen

Strings zusammensetzen

```
$location      = 'Bern';  
$temperatureC = 19.4;  
  
// Variante 1: Variablen in doppelten Anführungszeichen  
echo "Aare in $location: $temperatureC °C";  
  
// Variante 2: mit Punkt verbinden (Konkatenation)  
echo 'Aare in ' . $location . ': ' . $temperatureC . ' °C';
```

Doppelte Anführungszeichen setzen Variablen ein.

Einfache nicht: `'Hallo $name'` gibt wörtlich `Hallo $name` aus.

`\n` erzeugt einen Zeilenumbruch – funktioniert nur in doppelten Anführungszeichen.

Funktionen

Einmal schreiben, beliebig oft verwenden

Warum Funktionen?

Dieselbe Meldung für drei Orte – dreimal fast identischer Code:

```
echo "Aare in Bern: 19.4 °C um 10:00 Uhr.";
echo "Aare in Brienz: 16.8 °C um 10:00 Uhr.";
echo "Aare in Thun: 18.2 °C um 10:00 Uhr.";
```

- Änderst du das Format, musst du es an **drei** Stellen ändern
- Bei 500 Messwerten ist das unmöglich
- Eine Funktion beschreibt die Regel **einmal** und du rufst sie beliebig oft auf

Deklarieren und aufrufen

```
// 1. Deklarieren: die Regel einmal beschreiben
function formatMeasurement($location, $temperatureC, $measuredAt)
{
    return "Aare in $location: $temperatureC °C um $measuredAt Uhr.";
}

// 2. Aufrufen: so oft du willst, mit unterschiedlichen Werten
echo formatMeasurement('Bern', 19.4, '10:00');
echo formatMeasurement('Brienz', 16.8, '10:00');
```

- `function` + **sprechender Name** + Klammern
- **Parameter** sind Platzhalter, **Argumente** die konkreten Werte beim Aufruf
- Eine Funktion erledigt genau **eine** Aufgabe

Default-Werte und return

Default-Wert

```
function multiply($x, $y = 2)
{
    return $x * $y;
}

multiply(10);           // 20
multiply(10, 5);        // 50
```

return ist nicht echo

```
// gibt den Wert zurück
$msg = formatMeasurement(...);

// schreibt in die Antwort
echo $msg;
```

`return` liefert einen Wert zurück und **beendet** die Funktion sofort.

Wer nur `echo` nutzt, kann das Ergebnis später nicht weiterverarbeiten – und genau das brauchen wir in ETL.

Bedingungen

Entscheiden, was mit einem Wert passiert

Vergleichen

Ein Vergleich ergibt immer `true` oder `false`.

Operator	Bedeutung
<code>===</code>	ist gleich
<code>!==</code>	ist nicht gleich
<code><</code>	ist kleiner als
<code><=</code>	ist kleiner oder gleich
<code>></code>	ist grösser als
<code>>=</code>	ist grösser oder gleich

```
$x = 1;  
$y = 2;  
  
var_dump($x === $y); // false  
var_dump($x !== $y); // true  
var_dump($x < $y);   // true  
var_dump($x >= $y);  // false
```

== lose

Vergleicht nur den **Wert**.

PHP wandelt Typen automatisch um.

```
var_dump(5 == "5");      // true
var_dump(0 == false);    // true
var_dump("0" == false);  // true
var_dump(null == false); // true
```

Führt zu Überraschungen, die man beim Debuggen kaum findet.

=== strikt

Vergleicht **Wert und Typ**.

Keine automatische Umwandlung.

```
var_dump(5 === "5");      // false
var_dump(0 === false);    // false
var_dump("0" === false);  // false
var_dump(5 === 5);        // true
```

Im Kurs immer `===` und `!==` verwenden.

Bedingungen verknüpfen

```
$temperatureC = 19.4;  
$isOfficial   = true;  
  
// && – beide müssen wahr sein  
var_dump($temperatureC >= 16 && $temperatureC < 20); // true  
  
// || – eine von beiden reicht  
var_dump($temperatureC > 25 || $isOfficial === true); // true
```

– `&&` = **und** · `||` = **oder**

if / elseif / else

REGEL ZUERST IN ALLTAGSSPRACHE

- unter 16 °C → **kalt**
- ab 16, unter 20 °C → **frisch**
- ab 20 °C → **warm**

Erst die Regel, dann der Code – immer.

Die Bedingungen werden **von oben nach unten** geprüft.

Deshalb braucht der mittlere Fall kein zusätzliches `>= 16`.

```
function classifyTemperature($temperatureC)
{
    if ($temperatureC < 16) {
        return 'kalt';
    } elseif ($temperatureC < 20) {
        return 'frisch';
    } else {
        return 'warm';
    }
}
```

Kurzformen

switch – viele feste Fälle

```
$dataStatus = 'checked';

switch ($dataStatus) {
    case 'new':      echo 'neu'; break;
    case 'cleaned':  echo 'bereinigt'; break;
    case 'checked':  echo 'geprüft'; break;
    case 'published': echo 'veröffentlicht'; break;
    case 'archived': echo 'archiviert'; break;
    default:         echo 'unbekannt'; break;
}
```

? : – zwei Fälle in einer Zeile

```
echo $temperatureC >= 20
    ? 'warm'
    : 'nicht warm';
```

Praktisch, aber schnell unleserlich.

Im Zweifel `if` schreiben.

Beides ist **Referenzwissen** – im Cheatsheet `03_bedingungen.md` nachschlagen, wenn du es brauchst.

Arrays

Viele Werte in einer Variablen

Was ist ein Array?

Bisher: eine Variable, ein Wert.

Für Daten brauchen wir **eine Variable, viele Werte**.

```
// mühsam
$temperature1 = 18.9;
$temperature2 = 19.4;
$temperature3 = 20.1;

// gut
$temperatures = [18.9, 19.4, 20.1];
```

Jeder Datensatz, den ihr im Projekt aus einer API oder CSV-Datei ladet, kommt als Array in PHP an.

Arrays sind deshalb der wichtigste Datentyp des Kurses.

Indexiert oder assoziativ

Indexiert – Reihenfolge zählt

```
$temperatures = [18.9, 19.4, 20.1];  
  
echo $temperatures[0]; // 18.9  
echo $temperatures[1]; // 19.4
```

Die Zählung beginnt bei **0**.

Faustregel: **Liste** von gleichartigen Dingen → indexiert.

Ein Ding mit benannten Eigenschaften → assoziativ.

Assoziativ – Schlüssel zählt

```
$measurement = [  
    'location'    => 'Bern',  
    'temperature_c' => 19.4,  
    'measured_at' => '10:00',  
];  
  
echo $measurement['location']; // Bern
```

Die Form, die viele Daten haben

Eine **Liste** (indexiert) von **Datensätzen** (assoziativ):

```
$measurements = [  
  ['time' => '08:00', 'temperature_c' => 18.9],  
  ['time' => '10:00', 'temperature_c' => 19.4],  
  ['time' => '12:00', 'temperature_c' => 20.1],  
];  
  
echo $measurements[1]['temperature_c']; // 19.4
```

Genau so sehen JSON aus einer API, Zeilen aus einer CSV-Datei und Zeilen aus der Datenbank aus.

Wer diese Struktur lesen kann, kann alle drei lesen.

Mit Arrays arbeiten

```
$temperatures[] = 20.3;           // Wert am Ende anhängen
$stations['Interlaken'] = 17.5;   // Eintrag mit Schlüssel ergänzen

echo count($temperatures);        // Anzahl Einträge
var_dump($stations);              // ganze Struktur prüfen
```

Als Nächstes folgen vier wichtige Funktionen zum Umformen, Filtern, Zusammenfassen und Sortieren.

array_map – jeden Wert umformen

`array_map` wendet dieselbe Funktion auf jeden Wert an.

```
$temperatures = [18.9, 19.4, 20.1];

function formatTemperature($temperatureC) {
    return "$temperatureC °C";
}

$labels = array_map('formatTemperature', $temperatures);

print_r($labels);
```

ERGEBNIS

```
Array
(
    [0] => 18.9 °C
    [1] => 19.4 °C
    [2] => 20.1 °C
)
```

Die Anzahl Einträge bleibt gleich – nur die Werte ändern sich.

array_filter – passende Werte auswählen

`array_filter` behält nur Werte, welche die Bedingung erfüllen.

```
$temperatures = [18.9, 19.4, 20.1];  
  
function isWarm($temperatureC) { return $temperatureC >= 19.4; }  
  
$warmTemperatures = array_filter($temperatures, 'isWarm');  
  
print_r($warmTemperatures);
```

```
Array  
(  
    [1] => 19.4  
    [2] => 20.1  
)
```

`array_filter` behält die ursprünglichen Schlüssel.

array_reduce – viele Werte zusammenfassen

`array_reduce` macht aus einer Liste einen einzelnen Wert.

```
$temperatures = [18.9, 19.4, 20.1];  
  
function addTemperature($sum, $temperatureC) {  
    return $sum + $temperatureC;  
}  
  
$total = array_reduce($temperatures, 'addTemperature', 0);  
  
echo $total;
```

ERGEBNIS

58.4

`$sum` trägt das bisherige Ergebnis zum nächsten Wert weiter.

sort – Werte aufsteigend ordnen

`sort` sortiert die Werte vom kleinsten zum grössten.

```
$temperatures = [20.1, 18.9, 19.4];  
  
$sortedTemperatures = $temperatures;  
sort($sortedTemperatures);  
  
print_r($sortedTemperatures);
```

AUSGABE

```
Array  
(  
    [0] => 18.9  
    [1] => 19.4  
    [2] => 20.1  
)
```

`sort` verändert das übergebene Array – deshalb kopieren wir die Liste zuerst.

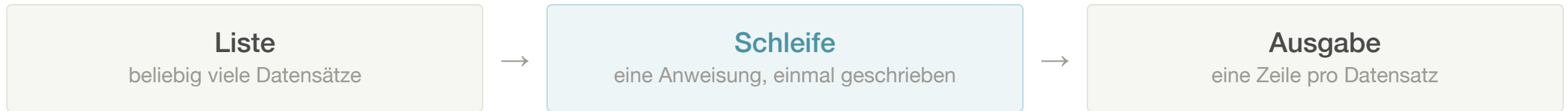
Schleifen

Eine Anweisung, beliebig viele Daten

Warum Schleifen?

Fünf Messungen von Hand ausgeben ist mühsam.

Ein Jahr mit einem Messwert pro Stunde ergibt **8760 Werte** – von Hand auszugeben ist unmöglich.



Eine Schleife ist erst dann korrekt, wenn sie mit **null**, **einem** und **vielen** Datensätzen sinnvoll läuft.

foreach – die wichtigste Schleife

```
foreach ($measurements as $measurement) {  
    echo $measurement['time'] . ': '  
        . $measurement['temperature_c'] . " °C\n";  
}
```

AUSGABE

```
08:00: 18.9 °C  
10:00: 19.4 °C  
12:00: 20.1 °C  
14:00: 20.3 °C  
16:00: 20.0 °C
```

- `$measurements` ist die **ganze Liste** (Plural)
- `$measurement` ist **ein Datensatz** (Singular)
- Kommt ein Datensatz dazu, musst du **nichts** ändern

Die anderen Schleifen

for – wenn du zählst

```
for ($i = 0; $i < 10; $i++) {  
    echo $i;  
}
```

Startwert · Abbruchbedingung · Schrittweite

while – solange etwas gilt

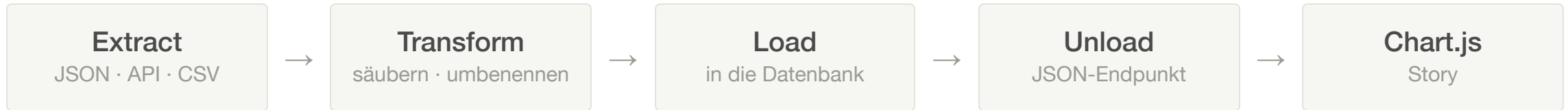
```
$i = 0;  
while ($i < 10) {  
    echo $i;  
    $i++;  
}
```

Wird die Bedingung nie `false`, läuft die Schleife ewig.

Zusammenspiel & Ausblick

Wozu das alles gut ist

Der rote Faden



In **jedem** dieser Schritte steckt, was du heute gelernt hast:

- **Arrays** – jede Datenquelle wird zu einem Array
- **foreach** – jeder Datensatz wird einzeln verarbeitet
- **Funktionen** – eine Regel pro Aufgabe, wiederverwendbar
- **Bedingungen** – fehlende oder unplausible Werte abfangen
- **Variablen & Typen** – `"19.4"` oder `19.4` entscheidet über das Ergebnis

Häufige Fehler

- **Weisse Seite:** fast immer ein PHP-Fehler – Fehlermeldungen einschalten oder Log lesen
- **Semikolon** am Zeilenende vergessen
- **\$** vor der Variablen vergessen
- **Code als Text sichtbar:** Datei per Doppelklick geöffnet statt über `http://`
- **Index beginnt bei 0:** `$a[1]` ist der zweite Wert
- **Tippfehler im Schlüssel:** `'temperature_c'` $\neq$ `'temperatureC'`
- **Einfache Anführungszeichen** setzen keine Variablen ein
- `==` statt `===` verglichen

Vorgehen bei jedem Fehler: **var_dump** vor die kritische Zeile setzen und schauen, was tatsächlich drin steht – nicht raten.

Was jetzt kommt

Code-Alongs

- `01_variablen`
- `02_funktionen`
- `03_bedingungen`
- `04_arrays`
- `05_schleifen`

Danach

- Digitale Übung `06_staedtevergleich`
- Zusatzübungen `01` – `05` zum Vertiefen
- Cheatsheets `A0` – `A5` zum Nachschlagen

Meilenstein M2: Am Ende des Tages steht eure eigene Datenfrage.

Zum Mitnehmen

PHP läuft auf dem Server
und liefert Daten.

Variablen, Funktionen, Bedingungen, Arrays und Schleifen – mehr PHP brauchst du für den ganzen ETL-Prozess nicht.

Nächster Schritt: Code-Along 01 – Variablen